



COMPañÍA MUNDIAL DE SEGUROS

GERENCIA DE TECNOLOGÍA - DIRECCION DE ARQUITECTURA DE SOLUCIONES DE SOFTWARE

Lineamientos APIs RESTful

Control del documento

Historia de cambios

Versión	Fecha	Referencia de Cambio	Elaborado Por
1.0	2024-11-14	Creación del Documento	Eduardo Ñustez Rodríguez
2.0	2024-11-15	Actualización apartado de seguridad	Leonardo Wilches Marcelo

Revisiones

Versión	Fecha	Revisado Por	Cargo /Área
1.0		Leonardo Wilches Marcelo	Director de Arquitectura

- [COMPañÍA MUNDIAL DE SEGUROS - GERENCIA DE TECNOLOGÍA - DIRECCION DE ARQUITECTURA DE SOLUCIONES DE SOFTWARE](#)
 - [Lineamientos APIs RESTful](#)
 - [Control del documento](#)
 - [Historia de cambios](#)
 - [Revisiones](#)
 - [Introducción](#)
 - [1.1. Objetivo](#)
 - [1.2. Alcance](#)
 - [1.3. Público objetivo](#)
 - [2. Líneamientos](#)
 - [2.1. Nombramiento de recursos y campos](#)
 - [2.2. Uso de Métodos HTTP](#)

- [2.3. Respuestas de la API](#)
- [2.4. Versionado](#)
- [2.5. Paginación y Filtrado](#)
- [2.6. Seguridad](#)
- [2.7. Documentación](#)
- [2.8. Lineamientos adicionales](#)

Introducción

1.1. Objetivo

El presente documento describe los lineamientos de Seguros Mundial para el diseño, construcción y consumo de APIs REST.

1.2. Alcance

Este documento describe los lineamientos técnicos para el diseño, construcción y consumo de APIs REST considerando aspectos como:

1.3. Público objetivo

Este documento está dirigido a arquitectos de soluciones y equipos de desarrollo que están involucrados en el diseño, desarrollo, implementación y mantenimiento de APIs RESTful. También es relevante para gerentes de proyectos y líderes técnicos que supervisan proyectos de desarrollo de software y necesitan asegurar que las APIs sean robustas, escalables y fáciles de mantener.

2. Líneamientos

2.1. Nombramiento de recursos y campos

- **Usar sustantivos en plural y en minúsculas:** Los recursos deben ser nombrados utilizando sustantivos, no verbos. Estos Ejemplo: /usuarios en lugar de /getUsuario.
- **Nomenclatura recursos con nombres extensos:** Para recursos de dos o más palabras utilizar nomenclatura Kebab case la cual usa el guión para combinar las palabras, por ejemplo: /items-pedido
- **Consistencia en nombramiento de recursos:** Los nombres de los recursos deben ser consistentes y reflejar la estructura de la aplicación. Ejemplo: /usuarios/{id}/ordenes para obtener las órdenes de un usuario específico.
- **Nomenclatura de campos:** Usar nomenclatura lowerCamelCase para todos los campos de entrada y salida de una solicitud. Por ejemplo:

```
{  
  "primerNombre": "John"  
}
```

2.2. Uso de Métodos HTTP

- **GET:** Para recuperar datos. Ejemplo: GET /usuarios/123.
- **POST:** Para crear nuevos datos. Ejemplo: POST /usuarios.
- **PUT:** Para actualizar completamente datos existentes. Ejemplo: PUT /usuarios/123.
- **PATCH:** Para actualizar completamente datos existentes. Ejemplo: PUT /usuarios/123.
- **DELETE:** Para eliminar datos. Ejemplo: DELETE /usuarios/123.

2.3. Respuestas de la API

- **Códigos de estado HTTP:** Usar códigos de estado significativos. Ejemplo:
 - **200 OK** para solicitudes que se resuelven de forma satisfactoria.
 - **201 Created** cuando se crea un nuevo recurso como resultado. Aplica para método **POST**
 - **204 No Content** indica que la solicitud ha sido procesada con éxito, pero el servidor no devuelve ningún contenido en la respuesta. Aplica para métodos **POST**, **PUT**, **PATCH** y **DELETE**.
 - **400 Bad Request** La solicitud no pudo ser entendida por el servidor debido a una sintaxis incorrecta.
 - **401 Unauthorized:** la solicitud no ha sido resuelta porque no tiene credenciales de autenticación válidas para el recurso solicitado.
 - **404 Not Found:** para recurso no encontrado.
 - **500 Internal Server Error:** El servidor encontró una condición inesperada que le impidió cumplir con la solicitud.
 - **503 Service Unavailable:** El servidor no está disponible temporalmente, generalmente debido a mantenimiento o sobrecarga.
- **Estructura de la respuesta:** La respuesta debe contener los siguientes campos:
 - **success:** Booleano que indica si se proceso exitosamente la petición
 - **timestamp:** Fecha y hora de la respuesta en formato Unix
 - **correlationId:** Id de correlación que identifica la solicitud
 - **data:** Objeto incluido en la respuesta
 - **estadoTransaccion:** Código y descripción del estado de la solicitud
 - **errors:** Arreglo con todos los errores de la solicitud

Un ejemplo de una respuesta es:

```
{
  "output": {
    "success": true,
    "timestamp": 1731624089,
    "correlationId": "96c153f0-a2d9-11ef-8e5e-02977ae602a9",
    "data": {
      "id": "025c7201-a90a-46f7-ad12-e26cf03e2671",
      "nombre": "John",
      "apelllidos": "Doe"
    },
    "estadoTransaccion": {
      "codEstado": "200",
      "descEstado": "OK"
    }
  },
}
```

```
    "errors": []
  }
}
```

- **Mensajes de error descriptivos:** Incluir un mensaje de error claro y conciso en el cuerpo de la respuesta. Ejemplo:

```
{
  "errors": [
    {
      "errorCode": "ERR001",
      "error": "El campo nombre no acepta valores numéricos"
    }
  ]
}
```

```
}
```

2.4. Versionado

- **Incluir la versión en el URI:** Esto ayuda a mantener la compatibilidad con clientes existentes mientras se introducen nuevas características. Ejemplo: `GET /v1/usuarios/123`.

2.5. Paginación y Filtrado

- **Parámetros de consulta:** Usar parámetros de consulta para manejar grandes cantidades de datos. Ejemplo: `GET /usuarios?limit=10&offset=30` para recuperar 10 usuarios comenzando desde el usuario número 30.

2.6. Seguridad

- **SSL:** Se debe usar SSL/TLS 1.2 o superior para cifrado de comunicación en transito entre el cliente y el servidor.
- **Autenticación y autorización:** Para proteger los recursos expuestos por las APIs puede usar los siguientes protocolos
 - **OAuth2** - Flow Client credentials, para comunicacion maquina a maquina desde origenes externos, el token generado debe ser formato JWT.
 - **Api-key** : Cabeceras de api key con Hash minimo de 32 carateres, alfanumerico o Hexadecimal, estos keys deben tener una caducidad periodica y solo ser usados para comunicacion maquina a maquina.
 - **ClientId Enforcement:** Se permite enviar cabeceras client-id y client-secret , en todas las peticiones, para comunicacion maquina a maquina.
- **Autorizacion** Los recuros expuestos por las apis deben estar con alcence limitado por roles y scopes, esto aplica con autenticacion Oauth2.0 Flow Client credentials.

2.7. Documentación

- **Documentar API:** Proporciona documentación clara y detallada para que los desarrolladores puedan entender y usar la API fácilmente.
- **Descripción de la API:** Proporcionar una visión general de lo que hace la API y su propósito.
- **Casos de Uso:** Incluir ejemplos de cómo y cuándo usar la API.
- **Lista de Endpoints:** Detallar todos los endpoints disponibles con sus respectivas rutas.
- **Métodos HTTP:** Especificar los métodos HTTP soportados (GET, POST, PUT, PATCH, DELETE) para cada endpoint.
- **Parámetros:** Describir los parámetros requeridos y opcionales para cada endpoint, incluyendo sus tipos y restricciones.
- **Ejemplos de Solicitudes y Respuestas:** Proporcionar ejemplos de solicitudes completas, incluyendo encabezados y cuerpos de solicitud. Incluir ejemplos de respuestas exitosas y de error, con códigos de estado HTTP y cuerpos de respuesta.

2.8. Lineamientos adicionales

- **Formato de fecha:** Usar ISO 8601, en UTC.
 - Para solo fechas, el formato debe ser yyyy-MM-dd, por ejemplo: "2016-01-27"
 - Para fechas completas, el formato debe ser yyyy-MM-ddTHH:mm:ssZ, por ejemplo: "2016-01-27T10:00:00Z"
- **Usar UTF-8:** Esperar caracteres acentuados o comillas en la salida de la API, aún cuando no se espere.